



# Statute Law Retrieval & Question Answering

**Nguyen Tan Minh**

Viện Khoa học và Công nghệ Tiên tiến Nhật Bản (JAIST)

Ngày 22 tháng 06 năm 2026



## Truy xuất văn bản pháp luật (Retrieval)

- Phương pháp Lexical (TF-IDF & BM25)

- Phương pháp Dense (Bi-encoder)

- Cross-encoder (Re-ranking)

- Truy xuất lai (Hybrid Retrieval) & RRF

- Tổng hợp các phương pháp Retrieval

## Hỏi đáp pháp luật (Question Answering)

- Xử lý đầu vào (Input Processing - Chunking)

- Phương pháp luận Hỏi đáp (QA - Methodology - Extractive, Generative)

- RAG nâng cao (Advanced RAG - HyDE, Query Expansion)

## TF-IDF

Biểu diễn văn bản qua vector trọng số từ:

$$\text{TF-IDF}(t, d) = \text{tf}(t, d) \times \log \frac{N}{\text{df}(t)}$$

- ▶ Không mô hình hoá thứ tự hay ngữ nghĩa.
- ▶ Yếu với từ đồng nghĩa (OOV).
- ▶ Nền tảng lý thuyết cho BM25.

## BM25

Cải tiến TF-IDF: saturation + chuẩn hoá độ dài:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1(1 - b + b \frac{|d|}{\text{avgl}})}$$

- ▶ Baseline cạnh tranh trên BEIR, COLIEE.
- ▶ Mạnh với thuật ngữ pháp lý *exact-match*.
- ▶ Triển khai: Elasticsearch, rank\_bm25.

## Hạn chế với luật tiếng Việt (Vocabulary Mismatch)

Bỏ sót khi truy vấn dùng ngôn ngữ tự nhiên khác biệt với văn bản luật chính thức:

- ▶ “ly hôn”  $\neq$  “chấm dứt hôn nhân”.
- ▶ “giật điện thoại”  $\neq$  “cướp giật tài sản”  $\Rightarrow$  Cần kết hợp Dense Retrieval.

# Retrieval — Dense (Bi-encoder)

## Cơ chế hoạt động

Kiến trúc **Song tháp (Dual-Tower)**:

- ▶ Mã hoá **độc lập** Câu hỏi ( $q$ ) và Đoạn luật ( $d$ ) thành 2 vector riêng lẻ.
- ▶ Tính độ tương đồng bằng góc giữa 2 vector:  $\cos(\mathbf{E}_q, \mathbf{E}_d)$ .

**Trực giác**: Giống như tóm tắt nội dung 2 văn bản rồi so sánh 2 bản tóm tắt đó.

## Ưu điểm

- ▶ Hiểu ngữ nghĩa, từ đồng nghĩa, diễn đạt vòng vo.
- ▶ **Cực nhanh**: Index offline sẵn, tìm kiếm thời gian thực qua Vector DB (Qdrant, FAISS).

## Mô hình & Thực tế

- ▶ **Tiêu biểu**: Sentence-BERT, BGE-M3, mE5, LLM2Vec.
- ▶ **Thực tế**: BGE-M3 hỗ trợ rất tốt tiếng Việt.

## Hạn chế

Nén cả đoạn luật thành 1 vector làm mất chi tiết từ khóa. Yêu khi khớp chính xác số điều/khoản (ví dụ: “khoản 2 điều 3”).

# Retrieval — Cross-encoder (Re-ranking)

## Cơ chế hoạt động

### Kiến trúc Đơn tháp (Single-Tower):

- ▶ Ghép chung Câu hỏi + Đoạn luật làm đầu vào mô hình:  $\text{score} = \text{BERT}(q \oplus d)$ .
- ▶ Cho phép tất cả từ tương tác chéo tự do (**Cross-Attention**).

**Thực giác:** Giống như giám khảo đọc cực kỳ kỹ từng chữ của cả câu hỏi và điều luật cùng lúc để chấm điểm.

## Mô hình tiêu biểu

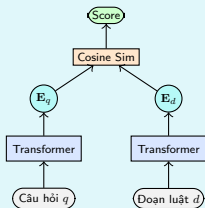
- ▶ bge-reranker-v2-m3 (Mạnh mẽ, đa ngôn ngữ).
- ▶ Qwen3-Reranker (LLM-based reranker).

## Hạn chế & Chiến lược

- ▶ **Rất chậm:** Không thể tính toán trước (offline index).
- ▶ **Chiến lược:** Chỉ dùng làm bộ lọc cuối (Re-ranker) xếp hạng lại Top-K ứng viên (ví dụ: rút từ 100 xuống 5).

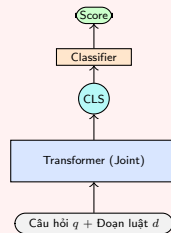
# Retrieval — So sánh Bi-encoder vs. Cross-encoder

## Bi-Encoder (Song tháp)



- ▶ **Cơ chế:** Query/Doc mã hóa riêng biệt.
- ▶ **Tương tác:** Chỉ ở lớp cuối (Cosine).
- ▶ **Tốc độ:** Rất nhanh, lưu index trước được.

## Cross-Encoder (Đơn tháp)



- ▶ **Cơ chế:** Query/Doc cùng đưa vào mô hình.
- ▶ **Tương tác:** Tự chú ý chéo (Cross-Attention).
- ▶ **Tốc độ:** Rất chậm, chạy trực tiếp lúc truy vấn.

## Ý tưởng cốt lõi [3]

Tương tác trễ ở cấp độ Token:

- ▶ Không nén cả đoạn văn thành 1 vector.
- ▶ Giữ lại vector của **từng từ (token)** trong câu.
- ▶ So khớp mỗi từ trong Câu hỏi với từ phù hợp nhất trong Luật, rồi tính tổng điểm (toán tử **MaxSim**).

**Thực giác:** Giống như dò từng từ khóa quan trọng của câu hỏi xem có xuất hiện trong luật không.

## Điểm cộng & Trừ

- ▶ **Chính xác:** Nhạy bén với số điều khoản (“khoản 2 điều 5”).
- ▶ **Vấn nhanh:** Nhờ mã hóa độc lập (giống Bi-encoder).
- ▶ **Tốn bộ nhớ:** Index rất nặng vì phải lưu vector của từng từ (30-40 GB / 1M docs).

## Giải pháp thực tiễn

**BGE-M3** có sẵn đầu ra ColBERT → Dễ tích hợp mà không cần dựng thêm hạ tầng riêng.

# Retrieval — Hybrid & Reciprocal Rank Fusion (RRF)

## Tại sao cần Hybrid?

- ▶ **BM25**: Mạnh với thuật ngữ pháp lý chính xác.
- ▶ **Dense**: Mạnh với câu hỏi diễn đạt vòng vo.
- ▶ Kết hợp hai nguồn  $\Rightarrow$  Bù đắp điểm yếu, bổ sung sức mạnh lẫn nhau.

## Score Fusion — Hạn chế

Kết hợp tuyến tính dựa trên điểm số (Score):

$$s = \alpha \cdot s_{\text{lexical}} + (1 - \alpha) \cdot s_{\text{dense}}$$

Bắt buộc phải chuẩn hoá thang điểm liên tục  $\Rightarrow$  Tính ổn định rất kém khi áp dụng chéo giữa các tập văn bản (corpus) khác nhau.

## Reciprocal Rank Fusion (RRF) [1]

Giải pháp của *Cormack et al. (2009)* — Hợp nhất dựa trên **thứ hạng (rank)** thay vì điểm số:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)} \quad (k = 60)$$

- ▶ Bền vững (Robust) với mọi phân phối điểm số.
- ▶ Triển khai dễ dàng (Elasticsearch, LangChain).

## Thực tiễn Pipeline VLQA

**BM25 + BGE-M3**

$\Downarrow$  *RRF*

Top-100 Candidates

$\Downarrow$  *BGE-Reranker*

**Top-5  $\Rightarrow$  Sinh câu trả lời (LLM)**



# Retrieval — Tổng hợp & So sánh

| Phương pháp   | Tốc độ      | Chất lượng | Đồng nghĩa | Triển khai     |
|---------------|-------------|------------|------------|----------------|
| BM25          | Rất nhanh   | Trung bình | ×          | Elasticsearch  |
| Bi-encoder    | Nhanh       | Tốt        | ✓          | FAISS, Qdrant  |
| ColBERT       | Trung bình  | Rất tốt    | ✓          | PLAID / BGE-M3 |
| Cross-encoder | Không index | Tốt nhất   | ✓          | Re-rank only   |
| Hybrid+RRF    | Nhanh       | Tốt nhất   | ✓          | Elasticsearch  |

## Khuyến nghị cho Vietnamese Law

- ▶ **Hybrid (BM25 + BGE-M3)** làm retriever chính
- ▶ **RRF** để fusion kết quả
- ▶ Re-rank bằng **bge-reranker-v2-m3**

### Nguyên tắc thiết kế

- ▶ Recall cao ở retrieval → Precision cao ở re-rank
- ▶ Tăng  $k$  khi câu hỏi liên quan nhiều điều luật
- ▶ Fine-tune bi-encoder trên dữ liệu pháp lý VN

# QA — Xử lý đầu vào (Chunking)

## Chiến lược Phân đoạn (Chunking)

**Mục đích:** Chia nhỏ văn bản luật dài thành các đoạn thông tin (chunks) vừa với giới hạn bộ nhớ ngữ cảnh của LLM.

- ▶ **Fixed-size Chunking (Cắt theo số từ):**  
Cắt cứng văn bản sau mỗi  $N$  ký tự/token.  
→ Dễ làm đứt gãy câu chữ, tách rời điều khoản, gây mất mát ngữ nghĩa pháp lý quan trọng.
- ▶ **Structural Chunking (Tách theo cấu trúc):**  
Tách tài liệu dựa trên hệ thống phân cấp logic của luật (Chương → Điều → Khoản → Điểm).  
→ Bảo toàn trọn vẹn văn cảnh và mối liên hệ suy luận.

## Ví dụ: Điều 173 BLHS

### Văn bản gốc (Khoản 1)

“...bị phạt cải tạo không giam giữ đến 03 năm hoặc phạt tù...”

### × Fixed-size Chunking

Chunk 1: ...phạt cải tạo không giam [CẮT]

Chunk 2: giữ đến 03 năm...

*Hậu quả: Đứt gãy khung hình phạt!*

### ✓ Structural Chunking

Chunk: [Tên Điều 173]

+ [Nội dung Khoản 1]

“...bị phạt cải tạo không giam giữ đến 03 năm...”

*Bảo toàn 100% ngữ cảnh pháp lý!*

# QA — Hỏi đáp dạng Trích xuất (Extractive QA)

## Khái niệm & Cơ chế

Dự đoán và trích xuất trực tiếp một đoạn văn bản liên tục (**text span**) từ tài liệu pháp luật làm câu trả lời:

$$\hat{a} = \text{BERT}(q, d)_{[\text{start} : \text{end}]}$$

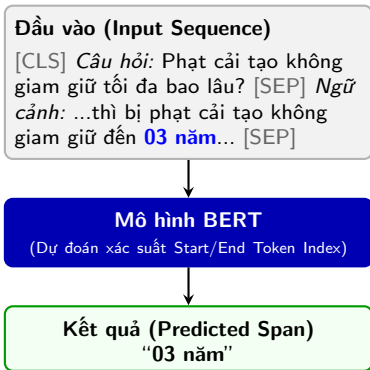
- ▶ **Mô hình tiêu biểu:** PhoBERT, XLM-RoBERTa (được fine-tune với lớp phân loại token đầu ra).
- ▶ **Tập dữ liệu VN:** ViQuAD, UIT-ViCoQA.
- ▶ **Vai trò:** Trích xuất bằng chứng pháp lý chính xác, có độ tin cậy tuyệt đối vì không sinh chữ (hallucination).

## Hạn chế cốt lõi

Hạn chế trước các truy vấn phức tạp đòi hỏi:

- ▶ Tổng hợp thông tin đa luồng (Multi-hop).
- ▶ Soạn thảo hợp đồng, tóm tắt tài liệu.
- ▶ Diễn giải lại bằng ngôn ngữ dễ hiểu.

## Kiến trúc Trích xuất (Span Extraction)



# QA — Hỏi đáp dạng Sinh (Generative QA / RAG)

## Retrieval-Augmented Generation

LLM tổng hợp câu trả lời dựa trên Câu hỏi ( $q$ ) và các Ngữ cảnh pháp luật ( $D$ ) được truy xuất:

$$A = \text{LLM}(q \oplus D_1 \oplus \dots \oplus D_k)$$

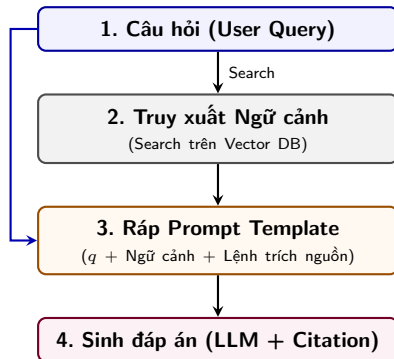
- ▶ **Mô hình tiêu biểu:** Qwen-3.5, GPT-OSS.
- ▶ **Giảm thiểu ảo giác (Hallucination):**
  - ▶ **Constraint:** Ép LLM chỉ suy luận dựa trên văn bản luật được cấp trong Prompt.
  - ▶ **Citation:** Bắt buộc trích dẫn nguồn gốc (Tên bộ luật, Điều, Khoản).

## Ví dụ

**Hỏi:** “Trộm xe máy 20 triệu bị phạt thế nào?”

**LLM:** “Theo **Khoản 1 Điều 173 Bộ luật Hình sự 2015**, hành vi trộm cắp tài sản từ 2 - 50 triệu đồng bị phạt cải tạo không giam giữ đến 3 năm hoặc phạt tù từ 6 tháng đến 3 năm.”

## Quy trình Hỏi đáp RAG



# QA — Trích xuất (Extractive) vs. Tạo sinh (Generative)

## 1. Trích xuất (Extractive QA)

Cơ chế: "Cắt" trực tiếp một đoạn văn bản (text span) từ tài liệu gốc làm đáp án.

$$\hat{a} = \text{BERT}(q, d)_{[\text{start} : \text{end}]}$$

- ▶ **Mô hình:** PhoBERT, XLM-RoBERTa (SQuAD, ViQuAD).
- ▶ **Ưu điểm:** Đơn giản, độ tin cậy tuyệt đối vì trích xuất nguyên văn, dễ kiểm soát nguồn.
- ▶ **Hạn chế cốt lõi:** Hạn chế khi đáp án pháp lý nằm rải rác ở nhiều khoản/điều khác nhau.

## Xu hướng thực tiễn

**RAG** là hướng tiếp cận chủ đạo hiện nay. Kỹ thuật Extractive dần lùi về đóng vai trò hỗ trợ (đánh dấu câu/đoạn trọng tâm).

## 2. Tạo sinh (Generative QA / RAG)

Cơ chế: "Đọc hiểu" nhiều tài liệu và tự viết lại một câu trả lời tổng hợp.

$$A = \text{LLM}(q \oplus D_1 \oplus \dots \oplus D_k)$$

- ▶ **Mô hình:** Qwen-3, GPT-OSS.
- ▶ **Ưu điểm:** Xử lý mượt mà các điều kiện chéo, ngoại lệ, và diễn giải lại luật bằng ngôn ngữ dễ hiểu.
- ▶ **Rủi ro:** Dễ sinh ra Ảo giác (Hallucination) nếu hệ thống truy xuất (Retrieval) cấp sai tài liệu.

# QA — Chiến lược Prompting

## Zero-shot

Instruction + context + câu hỏi, không có ví dụ mẫu.

- ▶ Không cần dữ liệu có nhãn
- ▶ Format đầu ra có thể không ổn định

## Few-shot

Cung cấp 2–4 cặp {hỏi, đáp} mẫu trước câu hỏi thật.

- ▶ Cải thiện format và độ chính xác
- ▶ Nhạy cảm với chất lượng của mẫu

## Chain-of-Thought

LLM giải thích từng bước dẫn chiếu điều luật trước khi kết luận.

- ▶ Cải thiện câu hỏi suy luận pháp lý
- ▶ Minh bạch, dễ kiểm chứng

## Ví dụ cấu trúc prompt cho Statute Law VN

[System] Bạn là trợ lý pháp lý. Chỉ trả lời dựa trên điều luật được cung cấp. Trích dẫn đúng số điều/khoản.

[Điều luật]  $\{D_1\} \dots \{D_k\}$     [Câu hỏi]  $\{q\}$

[Trả lời] Theo {điều X, khoản Y, Luật Z}, ...

## Cấu trúc lập luận IRAC [4]

Phương pháp lập luận tiêu chuẩn của luật sư:

- ▶ **I (Issue):** Xác định vấn đề pháp lý.
- ▶ **R (Rule):** Tìm quy định luật áp dụng.
- ▶ **A (Application):** Áp dụng luật vào tình huống.
- ▶ **C (Conclusion):** Đưa ra phán quyết cuối cùng.

## Minh họa Quy trình IRAC bằng ví dụ

- ▶ **Issue:** A lấy trộm xe máy trị giá 20 triệu đồng của B. A sẽ bị xử lý hình sự như thế nào?
- ▶ **Rule:** Điều 173 Bộ luật Hình sự quy định về tội trộm cắp tài sản. Khoản 1 áp dụng cho tài sản trị giá từ 2 đến dưới 50 triệu đồng.
- ▶ **Application:** Xe máy trị giá 20 triệu thuộc khung quy định tại Khoản 1 Điều 173.
- ▶ **Conclusion:** A có thể bị phạt cải tạo không giam giữ đến 03 năm hoặc phạt tù từ 06 tháng đến 03 năm.

---

<sup>1</sup><https://www.promptingguide.ai/>

# RAG nâng cao — Mở rộng truy vấn (Query Expansion)

## Mở rộng & Viết lại Truy vấn

- ▶ **Query Expansion:** Sử dụng LLM hoặc từ điển thuật ngữ để viết lại hoặc bổ sung thêm các từ đồng nghĩa, thuật ngữ pháp lý chuyên ngành vào câu hỏi ban đầu để nâng cao khả năng tìm kiếm (tăng Recall).
- ▶ **Giải quyết Vocabulary Mismatch:** Chuyển đổi ngôn ngữ dân dã của người dùng sang ngôn ngữ luật chính thống.

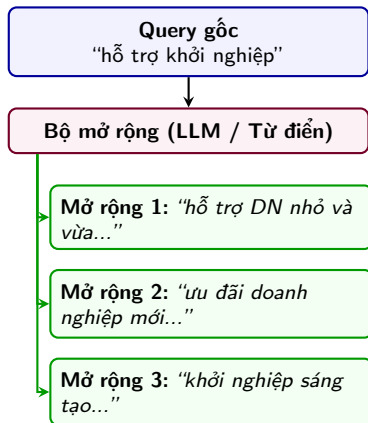
## Ví dụ minh họa

**Query gốc:** “hỗ trợ khởi nghiệp”

→ **Mở rộng:**

- ▶ “hỗ trợ doanh nghiệp nhỏ và vừa khởi nghiệp sáng tạo”
- ▶ “ưu đãi đối với doanh nghiệp mới thành lập”
- ▶ “Luật Hỗ trợ doanh nghiệp nhỏ và vừa”

## Sơ đồ Mở rộng Truy vấn





## Kỹ thuật HyDE [2]

- ▶ **HyDE (Hypothetical Document Embeddings):** LLM sinh ra một câu trả lời giả định (mang văn phong pháp lý) từ câu hỏi. Sau đó, lấy vector embedding của câu trả lời giả định này để đi tìm luật thật.
- ▶ **Tại sao cần HyDE:** Rút ngắn khoảng cách ngữ nghĩa giữa câu hỏi tự nhiên thô của người dùng và các điều luật trang trọng, khô khan.
- ▶ **Hạn chế:** Nguy cơ ảo giác lan truyền nếu tài liệu giả định sai lệch hoàn toàn thực tế.

## Ví dụ minh họa HyDE

**Hỏi:** “DN siêu nhỏ được ưu đãi thuế gì?” → **LLM sinh Hypothetical:** “Doanh nghiệp siêu nhỏ được áp dụng thuế suất thuế thu nhập doanh nghiệp thấp hơn có thời hạn so với thuế suất thông thường...” → **Mô hình truy xuất:** Tìm đúng Luật Hỗ trợ DN nhỏ và vừa 2017.

## Quy trình HyDE



- [1] Gordon V. Cormack, Charles L A Clarke, and Stefan Buettcher. “Reciprocal rank fusion outperforms condorcet and individual rank learning methods”. In: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. SIGIR '09. Boston, MA, USA: Association for Computing Machinery, 2009, pp. 758–759. ISBN: 9781605584836. DOI: 10.1145/1571941.1572114. URL: <https://doi.org/10.1145/1571941.1572114>.
- [2] Luyu Gao et al. “Precise Zero-Shot Dense Retrieval without Relevance Labels”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki. Toronto, Canada: Association for Computational Linguistics, July 2023, pp. 1762–1777. DOI: 10.18653/v1/2023.acl-long.99. URL: <https://aclanthology.org/2023.acl-long.99/>.

- [3] Omar Khattab and Matei Zaharia. “Colbert: Efficient and effective passage search via contextualized late interaction over bert”. In: *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 2020, pp. 39–48.
- [4] Fangyi Yu, Lee Quartey, and Frank Schilder. “Exploring the Effectiveness of Prompt Engineering for Legal Reasoning Tasks”. In: *Findings of the Association for Computational Linguistics: ACL 2023*. Ed. by Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki. Toronto, Canada: Association for Computational Linguistics, July 2023, pp. 13582–13596. DOI: 10.18653/v1/2023.findings-acl.858. URL: <https://aclanthology.org/2023.findings-acl.858/>.

# Xin trân trọng cảm ơn!



Thảo luận và Trả lời câu hỏi

*Statute Law Retrieval & Question Answering*

Open dataset coming soon

